CROSS / PROBLEM 01

Problem 1: Higher-throughput encrypted MNIST on eight TPU chips

CADathlon · `agent_pset` · AI agents allowed · ESSENTIAL problem · 100 points · proposed duration: 8 hours

1. Introduction

The FHE benchmarking suite of HomomorphicEncryption.org publishes a standardized ML-inference workload and a public results table. CROSS already runs that workload under CKKS on Google TPU v6e, correctly and at 128-bit security. It is not the fastest entry on that table.

Your task is to raise **throughput of the encrypted MNIST classifier across eight TPU v6e chips** while preserving its outputs, weights and cryptographic security. Unlike Problem 3, which optimizes one network on one chip, this track is about **how many images per second a fixed serving stack delivers**, and the headroom is mostly in how images are placed into ciphertexts.

The baseline leaves the ring nine-tenths empty. `make baseline-plan` reports:

```
{"depth": 3, "degree": 16384, "num_slots": 8192,
 "slots_used_by_input": 784, "slot_occupancy": 0.0957, ...}
```

One image occupies 784 of 8192 slots and the baseline puts one image in each ciphertext, so roughly 90% of every homomorphic operation is spent on zeros. Published submissions to the same benchmark reach far lower per-image cost by placing the **batch** on the ring axis instead of the feature axis. Closing that gap is the core of this problem; it is a packing and scheduling problem, not a model problem.

AI agents are allowed. You may use coding agents and AI assistants to design, implement, debug, test and profile your solution. Disclose the tools and assisted work in `REPORT.md`, and independently verify generated code and results. All correctness, privacy, security and scoring requirements apply.

2. Problem statement

2.1 The fixed network

Item	Value
Architecture	`Linear(784, 50)` → x^2 → `Linear(50, 10)`
Emitted depth	3

Item	Value
Data and client preprocessing	MNIST, 1×28×28; convert uint8 to [0,1], flatten to 784, then `(x-0.1307)/0.3081`
Client post-processing	argmax over the ten decrypted logits

The square is the activation: CKKS evaluates polynomials, and CROSS's activation registry accepts only a bare `x**(2**k)` squaring chain. The topology, widths, activation and preprocessing are fixed. Model weights are plaintext server constants; the privacy target is the client's image.

Organizers supply frozen trained weights and a hash manifest. The public fixtures in `test/` use seeded synthetic weights and are labelled so; they verify arithmetic, not classifier accuracy. Load **every** parameter from the official archive before a ranked run.

2.2 What to implement

Implement `build_service(model, input_shape, *, batch, devices)` in `src/bench_opt.py`. Return an object providing:

```
service.encrypt(images)   # (batch, 784) cleartext -> opaque ciphertext bundle
service.execute(bundle)   # ciphertext bundle -> ciphertext bundle
service.decrypt(bundle)   # ciphertext bundle -> (batch, 10) logits
service.release()         # free device memory
```

The bundle is deliberately opaque: it may be one ciphertext, a list, or any nesting of `Polynomial` values plus integer bookkeeping. **This is what lets you change the packing.** The baseline in `src/baseline.py` puts one image per ciphertext and one ciphertext per chip; you are not required to keep that.

`execute` runs on encrypted data only. It may not decrypt, call the cleartext model, inspect labels, cache by image identity or return precomputed answers. The local runner is a trusted test harness whose `Mapping` owns a context that also holds the secret key; evaluator code must not use that access. Include a source-level boundary audit in your report.

You may alter packing layout, plaintext-constant representation, BSGS planning, rotation reuse, kernel fusion, scheduling, device placement, memory reuse, batching across ciphertexts and exact modular kernels. The same optimization must work for arbitrary weights and inputs of the prescribed shapes. Edits to CROSS belong in `cross.patch`; edits under this problem's `src/` belong in the submission. You may add helpers. Do not change the benchmark, scoring, fixtures or checks.

You may not change weights, activation, input resolution, numerical preprocessing, topology, test data, ranked batch sizes or assigned devices. No pruning, retraining, approximate model replacement or plaintext fallback.

Cryptographic invariants. Keep a validated 128-bit `RingConfig`. The authored plan is depth 3, degree 16384, 8192 slots, 4 logical Q moduli, `dnum` 2, `log2(QP)` 365. A different secure ring is permitted **only** if `he_params.qp_is_secure` accepts it at 128-bit classical for the ring you actually run, and you print and report the derived parameters. Small insecure test rings receive no performance credit. Encryption noise must remain freshly sampled. Report `log2(Q)`, `log2(QP)`, the ceiling at your degree and the resulting margin.

Complete these tasks:

1. **P3.1 — establish the baseline:** verify provenance, run the cleartext and HE paths, and profile the baseline at each ranked batch on 1 and 8 chips.
2. **P3.2 — raise per-chip throughput:** attack slot occupancy or per-operation cost, and explain which operations or memory transfers it reduces.
3. **P3.3 — scale across chips:** show what your change does to 8-chip throughput and to scaling efficiency relative to one chip.
4. **P3.4 — compare and explain:** run all six ranked cases, preserve numerical and label accuracy, and report throughput with evidence.

Suggested starting points: place several images in the slot axis of one ciphertext; batch along the ring dimension so the summation runs across ciphertexts and needs no rotations; reuse rotation keys; eliminate repacks; fuse; shorten buffer lifetimes; reduce the client codec cost, which dominates small batches. `Mapping.estimate_live_memory()` is a logical estimate; use a profiler for actual peak HBM and label the two separately.

2.3 Inputs, outputs and six ranked workloads

The public `test1.in` – `test6.in` are JSON descriptions with batch size, device count, synthetic data seed and weight seed. `testN.out` holds real cleartext logits for every sample. The runner builds a deterministic weight set and checks shape, finite values, logit error and argmax agreement after HE. These fixtures use batches 1–8 on 1–4 devices to keep local development cheap; they are not the ranked suite.

All six ranked cases use TPU v6e chips from one host:

Case	Global batch	Chips	Mirrors
C1	1	1	single-image latency, one chip
C2	1	8	single-image latency, full host
C3	100	1	benchmark "small" instance, one chip
C4	100	8	benchmark "small" instance, full host
C5	1000	1	benchmark "medium" instance, one chip
C6	1000	8	benchmark "medium" instance, full host

C1 and C2 measure latency, where padding a batch of one across eight chips cannot help. C3–C6 measure throughput, where packing can. A single change that helps only one regime is a partial result; say so in the report.

2.4 Authored baseline observations

These were measured while authoring the package, on one host with eight TPU v6e chips, using the unmodified `baseline.py`. **They are indicative, not contest baselines**: the judge measures `T_base` on the official hardware with the frozen weights and images before the event, and those numbers govern scoring.

Measurement	Value
derived plan	depth 3, degree 16384, 8192 slots, 4 logical Q moduli, dnum 2
slot occupancy	784 / 8192 = **9.57%**
`log2(Q)` / `log2(QP)`	241.0 / 365.0, ceiling 438 at this degree, margin 73 bits
setup (build + keygen + compile), 1 chip	~126 s
end-to-end, batch 1, 1 chip	~180 ms, max abs logit error 1.2e-13
server execute only, batch 1, 1 chip	~31 ms
server execute only, batch 8, 8 chips	~4.5 ms per image

Two things to read off this table before optimizing. First, at batch 1 the end-to-end time is roughly six times the server execute time, so the client codec, not the accelerator, dominates the latency cases. Second, the server cost per image barely moves between one chip and eight when each chip holds one image, because the ring is doing the same mostly-empty work either way.

2.5 The ranked evaluation runs inside the benchmark suite

The six ranked cases are not scored by the local runner in `src/`. They are scored by the **FHE benchmarking suite's own harness**, at the commit pinned in `src/suite/suite_config.json`. That is what makes the result deterministic and comparable with the public results table: correctness is decided against the suite's frozen ground-truth labels and performance comes from its `Timing` block. Neither is self-reported by the submission.

`src/suite/install_suite.py` writes eight small shims into the suite's `submissions/mnist/` directory. Each one loads your `bench_opt.py` through the same `build_service` entrypoint the local runner uses, so there is one implementation and two ways to exercise it. The installer **refuses to run if `harness/` differs from the pinned commit** — the suite's rules forbid changing the harness, and this enforces that mechanically rather than by trust.

It does change two files outside your submission directory, and you should know which. `harness/utils.py::build_submission` unconditionally runs `scripts/get_openfhe.sh` and `scripts/build_task.sh`, which for a pure-Python submission would download LibTorch, build OpenFHE, and then fail on a CMake project that no longer exists. The installer replaces both with no-ops and keeps the originals as `.orig`. That is outside `harness/`, so it is within the suite's rules, and `make -C src suite-check` prints exactly what it replaced.

The harness runs each stage as a separate process, which has three consequences you must design for:

1. **Client and server share one key pair.** `build_service` receives it as `keys`; pass it to `Mapping(..., keys=keys)`. A service that generates its own keys will encrypt under one key and decrypt under another.

2. **Your ciphertext bundle crosses a file boundary.** `src/suite/bundle_io.py` serializes any nesting of `Polynomial`, `int`, `float`, `None`, `list`, `tuple` and `dict`, preserving each ciphertext's tracked CKKS scale. You do not implement this, but a bundle containing anything else is rejected.
3. **Build lazily.** Only the evaluation process needs the compiled `Mapping`; the client processes need the codec. `baseline.py` builds the context on first encrypt and the `Mapping` on first execute for exactly this reason. The resident server forces both during stage 3, with a warm-up pass, so the measured stage never pays for compilation.

Determinism: the suite selects images from `--seed 3`, the organizer publishes the seed, and each case runs `--num_runs 3` with the median as the statistic. Read that last part precisely. `run_submission.py` re-creates its RNG from the base seed inside the run loop, so all three runs derive the same `generate_input` seed (`3` → `1742692731`) and evaluate **the same images**, despite a comment in the harness that says otherwise. The three runs are therefore repeat timings on identical input: they measure machine noise, not input variation. Do not read a tight spread across them as evidence that your implementation generalizes.

One more consequence of the harness's structure: stages 1, 2.2 and 3 run **once**, outside the run loop. Every `results-N.json` repeats the same `Key Generation` and `Encrypted model preprocessing` values, and every `Total` carries that one-time setup. Only stages 4–10 — including `Encrypted computation`, the stage your score is built on — are measured per run. Setup is gated separately, at 1800 s, and is not charged per image.

```
make -C src suite-check                          # verify the pinned checkout
make -C src suite-install                        # write the stage shims
make -C src suite-baseline SIZE=1 P3_DEVICES=8   # unmodified baseline, C4
make -C src suite-test     SIZE=1 P3_DEVICES=8   # your bench_opt.py, C4
make -C src suite-gates                          # gate self-test, no accelerator
make -C src suite-score                          # deterministic rubric arithmetic
make -C src suite-stop                           # release the accelerator
```

Each run is archived before the next one overwrites it. The suite keys its output by instance alone — `measurements/<instance>/` and `io/<instance>/` — so C1 and C2, which share the single instance and differ only in chip count, would otherwise overwrite each other. `suite/archive.py` copies each case's three `results-N.json` files under `src/suite/results/<case>_<label>/` and freezes the harness's correctness evidence alongside them. `suite-baseline` and `suite-test` invoke it for you.

One thing the suite **cannot** give you is the cleartext reference for the accuracy gate. `harness/cleartext_impl.py` runs `harness/mnist/mnist_ffnn_model.pth`, a 784-128-64-10 network, while this problem's fixed model is the 784-50-10 network with the `x**2` activation. The harness's `Harness model quality` is therefore a different network's accuracy, and the agreement between it and your encrypted output measures how differently the two were trained, not whether your HE pipeline is faithful. Gating on it would charge you for the harness's training run. `archive.py` instead runs **this problem's model in cleartext** on the harness's own pixel file and compares that, in organizer-owned code your `bench_opt.py` never touches. The measured baseline makes the distinction concrete: on the 100 small-instance images it scores 96/100 in cleartext and 96/100 encrypted — identical — while the harness's larger network scores 97/100. The gap to the harness model is a model difference; the gap to your own cleartext run, which is what section 4 gates, is zero.

Freezing that evidence is necessary because two things are absent from `results-N.json`. The single instance has **no `Quality` block**: the harness checks it with `verify_result` under `check=False`, so a FAIL neither stops the run nor is recorded anywhere in the measurement file. And `calculate_quality` divides the number of correct

predictions by the number of predictions the submission wrote, so emitting fewer predictions than there are images yields a healthy-looking accuracy over the images that were skipped. `archive.py` recomputes both from the harness's own label and prediction files, and `score.py` refuses to award a case any points without that verdict. A case that writes an incomplete prediction set scores zero regardless of its speed.

`suite-score` then applies the rubric in section 4 against `src/suite/baseline_manifest.json` and prints one JSON verdict. It is explicitly **not** an official score: the judge reruns on exclusive hardware with its own baseline manifest.

The unmodified `baseline.py` measured through this path, median of three runs, on one host with eight TPU v6e chips. `src/suite/baseline_manifest.json` holds these numbers; they are what `suite-score` divides by, and they are indicative until the judge measures its own.

case	images	chips	stage 7	img/s	ms/image	server execute	ms/image
C1	1	1	0.1271 s	7.87	127.1	0.0325 s	32.5
C2	1	8	0.1329 s	7.52	132.9	0.0378 s	37.8
C3	100	1	3.4374 s	29.09	34.4	3.0463 s	30.5
C4	100	8	1.0276 s	97.31	10.3	0.6006 s	6.0
C5	1000	1	32.9975 s	30.31	33.0	30.4529 s	30.5
C6	1000	8	8.9178 s	112.14	8.9	5.5829 s	5.6

Three things in that table are the assignment. On one chip the server cost is flat at **30.5 ms per image** whether the batch is 100 or 1000, because the baseline walks the batch one lane at a time — it is sequential by construction. Eight chips buy **5.45×** at batch 1000 (30.45 s → 5.58 s), not 8×. And at batch 1 eight chips are slower than one (37.8 ms against 32.5 ms), because a single image cannot fill eight lanes and the cross-device work is pure overhead. Combine that with the 9.57% slot occupancy from section 2.4: the baseline leaves most of the ring and most of the machine idle.

Correctness is not the thing to optimize here — it is already exact. Across C3–C6 the encrypted output agreed with the same model in cleartext on **every one of the 1100 images**, and `own_cleartext_agreement` is 1.0 in all six cases. Your job is to keep that while making the table's right-hand columns smaller.

For orientation only, the suite ships the reference implementation's own measurements at the pinned commit, generated in February 2026 and copied to `src/suite/results/_upstream_reference/`. Median `Encrypted computation`:

instance	images	compute (s)	ms/image	accuracy
single	1	7.6145	7614.50	not reported for this instance
small	100	647.0703	6470.70	0.9600

instance	images	compute (s)	ms/image	accuracy
medium	1000	6505.6093	6505.61	0.9740

Those runs were made on the reference implementation's machine, which the repository does not describe, so the gap to the CROSS baseline mixes hardware with implementation and is **not** a measured speedup. Your score is computed against the CROSS baseline on contest hardware, never against this table.

Organizers supply 64 public calibration images and select 256 disjoint hidden test images, with 25 or 26 per class. Identical images and weights are used for the baseline and each candidate. Evaluate every image in a ranked batch without padding or dropping samples; where the batch exceeds 256 images, the organizer publishes the repeat schedule. Public boundary cases additionally include all zeros, uniform values, high-contrast pixels and distinct images in every lane. The harness may change seeds and weights for structural tests to catch specialization; ranked accuracy always uses the frozen trained weights.

For **each** case all logits must satisfy `abs(HE−clear) <= 0.02 + 0.01*abs(clear)`, at least **99%** of image argmaxes must match the cleartext reference, and label accuracy may fall by at most **1 percentage point** relative to that same model's cleartext predictions on the same images. "That same model" is literal and matters: it is this problem's 784-50-10 network in cleartext, not the harness's 784-128-64-10 reference, and not the harness's `Harness model quality` figure. On the suite path `archive.py` computes it; the logit gate applies on the local runner, which is the only path that sees cleartext logits rather than labels. Report absolute accuracy too. Require all three gates. NaN, Inf, a shape mismatch or a dropped image fails the case. Organizers validate that the unmodified baseline passes before release. Tolerances are contest choices, not guarantees from CKKS.

A note on precision if you change the ring: reducing the modulus is a legitimate optimization, but the logit error grows quickly as the chain shortens. The authored plan decrypts within about 2e-12 of cleartext; a 100-bit chain at degree 4096 measured about 2e-03 on the same network family. The gates above are absolute, so verify them at your chosen parameters rather than assuming the baseline's margin carries over.

---

3. How to compile and run

The organizer image contains Python, NumPy, PyTorch, JAX, the TPU runtime and the pinned CROSS checkout. `CROSS_ROOT` selects another checkout if needed. The six public reference tests run without training or downloading.

```
cd cadathlon/agent_pset/p1_benchmark/src
make all
make reference-test      # numerical fixtures only, no encryption
make baseline-plan       # actual CROSS plan and slot occupancy, no keygen
make baseline-test       # unmodified encrypted CROSS, test1; minutes/GB
make test                # all six cases using your bench_opt.py
python runner.py --case ../test/test6.in --baseline
make submit              # writes ../submission.tar.gz
```

The local commands above are for development. The ranked cases run through the benchmark suite (section 2.5):

```
cd cadathlon/agent_pset/p1_benchmark
make -C src suite-check                          # pinned checkout, harness intact
make -C src suite-install
make -C src suite-baseline SIZE=2 P3_DEVICES=8   # C6 baseline
make -C src suite-test     SIZE=2 P3_DEVICES=8   # C6 with your implementation
make -C src suite-gates
make -C src suite-score
make -C src suite-stop
```

`SIZE` selects the suite instance (0 single, 1 small, 2 medium) and `P3_DEVICES` the chip count; together they pick the ranked case. `MLINF_ROOT` points at the suite checkout if it is not `~/ml-inference`.

The editable TODO deliberately raises `NotImplementedError` until completed. `baseline.py` provides working reference code; copying it produces a valid starting point with no speedup. `make all` only checks Python syntax. Logs go to stderr and one machine-readable JSON result goes to stdout per case. Do not byte-compare noisy CKKS outputs to `.out`; use the stated bounds.

Runtime limits: up to eight TPU v6e chips on the fixed host image, 512 GiB maximum host RSS, and no device OOM. Prepare, key generation and compilation may take at most **30 minutes per configuration**. Each online batch may take at most **300 seconds**. A failed or timed-out case scores zero. CPU runs are useful for correctness but do not count toward the TPU score.

Prepare once per configuration and release it before building the next. Two untimed warm-up requests, then five measured runs on distinct batches from a fixed published selection. Fresh encryption every time. Synchronize output ciphertexts with `jax.block_until_ready` before stopping the server timer. Count dispatch, server computation, online host work and mandatory transfers; no lazy work may escape timing. Report median and p95 (nearest-rank) of the five samples. Ranked throughput `X = B / T`, where `T` is the median **end-to-end batch wall time** from client encryption through decryption for `B` images. Also report server-only timing and `T / B` seconds per image. Baseline and candidate use identical boundaries, images, ring and device counts.

Compilation, key generation and constant preparation are reported separately and excluded from warm `T`. They still obey the setup limit. Reuse only public weights, constants, keys and executables, never image-specific ciphertext intermediates. The judge runs baseline and candidate in separate clean processes, interleaves their reservations and reruns ties within 2% once. Record physical peak device memory and process RSS, plus logical estimates if used. Throughput obtained by changing the ranked batch or the assigned chip count does not count as an optimization.

---

4. How to score

Let `G_i` equal 1 only if case i passes all numerical, accuracy, privacy, parameter, memory and runtime gates, otherwise 0. Let `R_i = X_i / X_base_i` be the throughput ratio, using organizer-rerun medians. The score is:

`sum_i [5*G_i + 10*G_i*clamp(log2(R_i)/2, 0, 1)] + E`,

where i covers **all six cases** and E is the evidence score below. A correct baseline earns 30 execution points; a 2× improvement earns 5 speed points per passing case; 4× or more earns 10. Regressions earn no speed points. Correctness contributes 30, speed 60, evidence 10. The latency cases (C1, C2) and the throughput cases (C3–C6) are both scored, so an entry that only helps large batches forfeits up to 20 speed points.

Evidence E awards 2 points each for: checkpoint and environment reproducibility; baseline profiles at 1 and 8 chips; an explanation tied to measured bottlenecks, including slot occupancy before and after; complete raw timing and actual-memory records; and regression tests plus a privacy and parameter review that states `log2(QP)`, the ceiling and the margin.

`X_i` comes from the suite's own `results-{1,2,3}.json`, archived per case under `src/suite/results/`, never from a contestant-printed number. The accuracy and argmax gates come from `archive.py`, which recomputes them from the harness's label and prediction files and from this problem's model run in cleartext — organizer-owned code, not the submission's. `src/suite/score.py` performs the arithmetic and prints it as JSON, and covers the numerical, accuracy, completeness, setup and runtime gates. The privacy, parameter and memory gates in `G_i` are assessed by the judge from `REPORT.md` and a source audit; a case that fails any of them scores zero regardless of what the scorer printed.

Ties break by geometric mean throughput ratio across all six passing cases (an incomplete entry ranks below a fully passing one), then lower peak device memory, then lower total setup time. The local runner does not assign an official score from contestant-supplied times.

Submit `src/`, `cross.patch`, `REPORT.md`, raw result JSON and environment and weight hashes. Include both successful and failed configurations. Four report pages are sufficient; logs and code are additional. Do not submit datasets, trained weights, secret keys, caches, generated reference outputs or unrelated files. `make submit` packages the listed files; it does not upload them.

---

5. References

1. Tong et al., Leveraging ASIC AI Chips for Homomorphic Encryption, HPCA 2026; supplied `CROSS_paper.pdf` and paper. Use BAT/MAT and profiling ideas; paper numbers are not contest baselines.
2. FHE benchmarking suite, ML-inference workload: repository and published results. The results table shows what other submissions achieve on this workload and on what hardware; treat those numbers as context, not as contest baselines.
3. CROSS `jaxite_word/API_REFERENCE.md`, `nn.py`, `packing.py`, `mapping.py`, `bsgs.py`, `he_params.py`, `profiler.py`; `profile_analysis/analyze_trace_json.py`. `API_REFERENCE.md` documents the canonical rank-5 `Polynomial`, `global_batch`, device placement and the codec boundary.
4. CROSS tutorial: CROSS, TPU Novice to Master, NTT Algorithms and Open Challenge. Current canonical code takes precedence over older insecure-ring performance tables.